Faculty of Science
Software Engineering Teaching Unit
University of Kelaniya, Sri Lanka

ConstructPro ERP

Test Coverage Report

Sprint 8 unit test coverage

SENG 34213 – System Development Project

Team member	Student number
Iddamalgoda I.H.R.P.	SE/2021/038
Dhanushan K.I.	SE/2021/027
Ranasundara R.I.K.	SE/2021/001
Abeywardena M.B.	SE/2021/037

Deliverable for documents/testing/coverage-sprint8.pdf (guideline §10.1)

Supervisor: Dr. Nalin Warnajith
Client: Ishara Homes Pvt. Ltd.

October 2026

Chapter 1

Summary

This report records the unit test coverage of ConstructPro ERP at the end of Sprint 8, for Section 6.4 of the SENG 34213 guideline. Section 6.4 requires at least 80% coverage on new code and 90% branch coverage on critical business logic, and the team's coding standards (`documents/standards/project_standards.md`) repeat the 80% target.

The figures were produced on 23 October 2026 by running the unit suites with coverage on a local copy of each repository. The CI pipeline does not yet run coverage, so this is a local run rather than a CI artifact; the steps to add coverage to CI are in Chapter 7 of the team's “What We Missed” document.

Table 1.1: Coverage runs

Repository	Commit	Command	Result
backend	`f6bf853`	`jest --config jest.unit.config.js --coverage`	84 suites, 1,535 tests passed; per-file thresholds met
frontend	`4e80694`	`npm run test:coverage`	173 node:test tests passed
frontend	`4e80694`	`vitest run --coverage` with the quotation, dashboard and portal configurations	118, 12 and 16 tests passed

Table 1.2: Overall backend unit coverage

Metric	**Backend unit coverage**
Lines	76.4%
Statements	76.3%
Branches	71.5%
Functions	71.5%

Overall backend line coverage is 76.4%, below the 80% target. Nine of the fourteen backend applications meet it; the gaps are concentrated in the authentication, lead, gateway, calendar and user services and the shared library, whose main flows are covered by the 548 integration tests instead.

Chapter 2

Backend Coverage by Service

Table 2.1 groups the backend files by application or library. “Size” is the number of executable lines measured, and “Stmts” and “Funcs” are statement and function coverage.

Table 2.1: Backend unit coverage by service

Service or library	Lines	Stmts	Branches	Funcs	Size	≥ 80%
quotation-service	99.7%	99.8%	89.0%	100.0%	379	Yes
document-service	93.4%	92.9%	87.2%	99.0%	678	Yes
analytics-service	93.0%	92.9%	90.3%	99.0%	373	Yes
ai-service	92.7%	92.8%	91.8%	98.8%	412	Yes
invoice-service	92.0%	92.1%	88.9%	98.9%	450	Yes
client-service	91.2%	91.9%	81.2%	97.8%	204	Yes
task-service	89.6%	89.4%	85.6%	96.0%	289	Yes
payment-service	85.9%	85.8%	87.8%	97.1%	205	Yes
project-service	83.2%	83.3%	79.3%	75.6%	707	Yes
user-service	67.3%	67.9%	73.0%	51.4%	153	No
libs/common	63.6%	60.0%	76.6%	52.6%	225	No
calender-service	60.8%	60.8%	50.4%	38.3%	311	No
api-gateway	53.1%	53.9%	47.6%	46.0%	1024	No
lead-service	43.2%	42.6%	24.2%	29.4%	250	No
auth-service	35.7%	38.4%	36.6%	21.4%	157	No
libs/auth	0.0%	0.0%	0.0%	0.0%	21	No
libs/config	0.0%	0.0%	0.0%	0.0%	8	No
libs/contracts	0.0%	0.0%	100.0%	100.0%	24	No
libs/database	0.0%	0.0%	0.0%	0.0%	30	No

The shared libraries `libs/auth`, `libs/config`, `libs/contracts` and `libs/database` have no unit tests of their own (83 lines in total); they are exercised by the integration tests.

2.1 Critical business logic

Section 6.4 asks for 90% branch coverage on critical business logic, as specified in the SDS. Table 2.2 shows the files that handle money, quotation conversion, project progress and AI prediction. Focused Jest configurations in the backend already enforce 90% branches on the invoice, payment, invoice PDF and AI files.

Table 2.2: Branch coverage of critical business logic

File	Lines	Branches	≥ 90% branches
payment-service/src/payment.service.ts	98.7%	98.2%	Yes
ai-service/src/ai-provider.service.ts	100.0%	97.7%	Yes
invoice-service/src/invoice.service.ts	98.9%	96.6%	Yes
invoice-service/src/pdf/invoice-pdf.service.ts	100.0%	96.0%	Yes
quotation-service/src/quotation.service.ts	100.0%	95.9%	Yes
ai-service/src/ai-forecasting.service.ts	100.0%	94.7%	Yes
project-service/src/project.service.ts	96.2%	89.4%	No
project-service/src/milestone.service.ts	79.8%	73.6%	No

The milestone service, which computes weighted project progress, is the clearest gap: 73.6% of its branches are covered by unit tests, although its persistence and concurrency behaviour is covered by integration tests.

Chapter 3

Frontend Coverage

The frontend measures coverage per test suite rather than for the whole application. Table 3.1 shows each suite and the files it measures.

Table 3.1: Frontend coverage by suite

Suite	Tests	Lines	Stmts	Branches	Funcs
node:test, API and analytics modules	173	98.3%	–	70.0%	100.0%
Vitest, quotation module	118	94.6%	93.9%	85.8%	93.3%
Vitest, dashboard	12	75.4%	74.9%	66.4%	71.4%
Vitest, client portal	16	71.4%	70.8%	47.3%	73.0%

The node:test suite measures the dashboard, analytics and AI forecasting API modules; it reports line, branch and function coverage only. The dashboard and client portal suites are below the 80% target.

Before submission
Once the coverage step is in CI, replace these figures with the CI run used for the final submission and link that run in the Sprint 8 review.